Пермский институт (филиал) федерального государственного бюджетного образовательного учреждения высшего образования «Российский экономический университет имени Г.В. Плеханова»

Отделение информационных технологий и программирования

Практическая работа №2

Разработка WPF-приложения «Калькулятор» с модульным тестированием

Работу выполнил: Зеленин Арсений

Группа: ИПо-21

Преподаватель: Берестов Дмитрий Борисович

Пермь

2026

Оглавление

1. Цель и задачи работы 3
2. Объект тестирования 3
3. Архитектура решения 4
4. Ход выполнения работы 5
4.1. Создание архитектуры решения 5
4.2. Реализация бизнес-логики 5
4.3. Разработка пользовательского интерфейса 6
4.4. Написание модульных тестов 7
4.5. Тестирование базовой функциональности 7
4.6. Исправление выявленных дефектов 8
5. Результаты работы 9
6. Заключение 11

1. Цель и задачи работы

Целью работы являлась разработка настольного приложения-калькулятора на платформе .NET с графическим интерфейсом WPF, вынесенной в отдельную библиотеку бизнес-логикой и набором автоматизированных модульных тестов.

В ходе выполнения работы решались следующие задачи:

- проектирование трёхуровневой структуры решения с разделением ответственности между слоями;
- реализация ядра вычислений в виде библиотеки классов: разбор выражений, вычисление результата, работа с историей и настройками;
- разработка пользовательского интерфейса WPF с использованием подхода MVVM;
- написание набора модульных тестов на фреймворке xUnit;
- проверка приложения, выявление и устранение дефектов по результатам тестирования;
- оформление интерфейса: светлая и тёмная темы, локализация RU/EN, ввод с клавиатуры.

2. Объект тестирования

Центральным элементом системы является класс CalculatorEngine библиотеки ZeleninCalculatorCore. Он инкапсулирует вычисление выражений и работу с сохраняемыми данными:

- Calculate(string expression) — вычисляет выражение и добавляет запись в историю;
- Evaluate(string expression) — вычисляет выражение без записи в историю (предпросмотр результата);
- GetHistory() и ClearHistory() — доступ к истории и её очистка;
- GetThemeSettings() и UpdateThemeSettings(...) — чтение и изменение настроек темы;
- SaveAll() — сохранение истории и настроек в файлы.

Разбор выражений выполняет статический класс ReversePolishNotation. Выражение разбивается на токены, переводится в обратную польскую запись (алгоритм сортировочной станции) и вычисляется в стековой машине. Поддерживаются операции +, −, ×, ÷, ^, скобки, унарный минус и десятичные числа, записанные как с точкой, так и с запятой. Операция ^ правоассоциативна (2^3^2 = 512), деление на ноль приводит к исключению DivideByZeroException, а переполнение типа decimal — к исключению OverflowException.

3. Архитектура решения

Работа выполнялась в среде Visual Studio 2022 на целевой платформе .NET 8. Решение ZeleninCalculator.sln состоит из трёх проектов:

1. ZeleninCalculatorCore — библиотека классов (net8.0), содержащая бизнес-логику. Внутри выделены папки Calculators, Models, Services и Infrastructure.

2. ZeleninCalculatorApp — WPF-приложение (net8.0-windows): файл MainWindow.xaml, модель представления MainViewModel.cs, вспомогательные классы RelayCommand.cs и StringToBrushConverter.cs.

3. ZeleninCalculatorTests — проект модульных тестов на xUnit 2.5.3 с классом CalculatorEngineTests.

Настройка зависимостей: приложение ссылается на Core, тестовый проект — на Core. Это позволяет проверять логику независимо от интерфейса. Пользовательские данные сохраняются в каталоге %LocalAppData%\ZeleninCalculator\ в файлах history.json и settings.json.

```
Calculator-Zelenin/
├── ZeleninCalculator.sln
├── ZeleninCalculatorCore/
│   ├── Calculators/    CalculatorEngine.cs, ReversePolishNotation.cs
│   ├── Infrastructure/ AppPaths.cs
│   ├── Models/         CalculationHistoryItem.cs, ThemeSettings.cs
│   └── Services/       HistoryService.cs, SettingsService.cs
├── ZeleninCalculatorApp/
│   ├── Infrastructure/ RelayCommand.cs, StringToBrushConverter.cs
│   ├── ViewModels/     MainViewModel.cs
│   └── MainWindow.xaml
```

```
└── ZeleninCalculatorTests/
    └── CalculatorEngineTests.cs
```

4. Ход выполнения работы

4.1. Создание архитектуры решения

На первом этапе было создано пустое решение ZeleninCalculator.sln, в которое добавлены три проекта: библиотека классов, WPF-приложение и проект тестов. Ссылки между проектами настроены командами:

```
dotnet new sln -n ZeleninCalculator
dotnet new classlib -n ZeleninCalculatorCore
dotnet new wpf -n ZeleninCalculatorApp
dotnet new xunit -n ZeleninCalculatorTests
dotnet sln add ZeleninCalculatorCore ZeleninCalculatorApp ZeleninCalculatorTests
dotnet add ZeleninCalculatorApp reference ZeleninCalculatorCore
dotnet add ZeleninCalculatorTests reference ZeleninCalculatorCore
```

4.2. Реализация бизнес-логики

Вычисление выражения начинается с нормализации строки, после чего вызывается разборщик, а результат записывается в историю:

```
public decimal Calculate(string expression)
{
    var normalized = NormalizeExpression(expression);
    var result = ReversePolishNotation.Evaluate(normalized);
    AddHistory(normalized, result);
    return result;
}
```

Токенизатор распознаёт числа, операторы и скобки. Для поддержки обоих разделителей дробной части запятая заменяется на точку, а разбор выполняется в инвариантной культуре:

```
var numberText = expression.Substring(start, i - start).Replace(',', '.');
if (!decimal.TryParse(numberText, NumberStyles.Number, CultureInfo.InvariantCulture,
    out var value))
    throw new FormatException("Invalid number: " + numberText);
```

История хранится в памяти и ограничена 200 записями: при превышении лимита самые старые записи удаляются. Метод Evaluate используется для предпросмотра результата и историю не изменяет.

4.3. Разработка пользовательского интерфейса

Интерфейс реализован по шаблону MVVM: представление описано в XAML, состояние и команды — в MainViewModel, связь между ними обеспечивается привязками данных. Окно разделено на две области.

Структура окна:

- шапка — бейдж с буквой Z в акцентном цвете, название приложения, подпись автора и кнопки переключения языка и темы;
- карточка дисплея — метка «Выражение», строка выражения, метка «Результат» и крупный результат, который пересчитывается по мере ввода;
- клавиатура — сетка 6×4: цифры, десятичная точка, операторы, скобки, степень, CE, C и клавиша «=»;
- правая панель — история вычислений в виде карточек; щелчок по карточке восстанавливает выражение.

Обработка событий. Каждой кнопке сопоставлена команда:

- AppendCommand, EvaluateCommand, BackspaceCommand, ClearEntryCommand, ClearAllCommand;
- ToggleThemeCommand, ToggleLanguageCommand, UseHistoryCommand, ClearHistoryCommand.

Ввод с клавиатуры обрабатывается в code-behind: PreviewTextInput принимает цифры и символы операций, PreviewKeyDown обрабатывает Enter, Backspace и Escape.

```
<Button Style="{StaticResource OpButtonStyle}"
        Command="{Binding AppendCommand}"
        CommandParameter="/" Content="÷" Margin="4"/>
```

```
private void OnPreviewTextInput(object sender, TextCompositionEventArgs e)
{
    if (DataContext is not MainViewModel vm || string.IsNullOrEmpty(e.Text))
        return;

    char ch = e.Text[0];
    if (char.IsDigit(ch) || ch is '.' or ',' or '+' or '-' or '*' or '/' or '^' or '('
    or ')')
    {
        vm.AppendToken(e.Text);
        e.Handled = true;
```

```
    }
}
```

Оформление. Используется тёмная тема с фоном #0E1220 и светлая с фоном #EEF2FC, акцентный цвет — #5B7CFA. Цвета хранятся в модели представления в виде строк и преобразуются в кисти конвертером StringToBrushConverter, поэтому смена темы выполняется без перезапуска приложения. Кнопки имеют скругление 12 px, карточки — 16 px, шрифт — Segoe UI Variable Display. Заголовок окна окрашивается в тёмный цвет функцией DwmSetWindowAttribute.

4.4. Написание модульных тестов

Тесты написаны на xUnit. Каждый тест создаёт экземпляр движка с временными файлами истории и настроек, поэтому тесты не зависят друг от друга и от данных пользователя:

```
private static CalculatorEngine CreateEngine()
{
    var root = Path.Combine(Path.GetTempPath(), "ZeleninCalculatorTests",
    Guid.NewGuid().ToString("N"));
    Directory.CreateDirectory(root);
    return new CalculatorEngine(
        historyFilePath: Path.Combine(root, "history.json"),
        settingsFilePath: Path.Combine(root, "settings.json"));
}
```

Параметризованные тесты реализованы через атрибуты [Theory] и [InlineData], что позволяет одной группой проверить несколько выражений:

```
[Theory]
[InlineData("2+3", 5)]
[InlineData("10-4", 6)]
[InlineData("7*8", 56)]
[InlineData("20/5", 4)]
[InlineData("2^10", 1024)]
public void Calculate_BasicOperations_ReturnCorrectResult(string expression, decimal
    expected)
{
    var engine = CreateEngine();
    Assert.Equal(expected, engine.Calculate(expression));
}
```

4.5. Тестирование базовой функциональности

Помимо базовых операций проверены:

- десятичные числа с точкой и запятой (1,5 + 2,5 = 4; 0,1 + 0,2 = 0,3);

- приоритет и ассоциативность операций (2 + 3 * 4 = 14; 2^3^2 = 512; 10 − 2 − 3 = 5);
- унарный минус (−5 + 2 = −3; 3 * −2 = −6; −2^2 = −4);
- возведение в нулевую и отрицательную степень (5^0 = 1; 2^(−2) = 0,25);
- ошибки: деление на ноль, переполнение, незакрытые скобки, пустое выражение, недопустимые символы;
- история: добавление, отсутствие записи при предпросмотре, ограничение в 200 записей, сохранение между запусками;
- настройки темы: значения по умолчанию, изменение, сохранение, проверка на null.

Общее количество тестов — 42. Запуск выполняется командой:

```
dotnet test ZeleninCalculator.sln -c Release
```

Результат выполнения:

```
Пройден! : не пройдено 0, пройдено 42, пропущено 0, всего 42,
           длительность 286 ms - ZeleninCalculatorTests.dll (net8.0)
```

4.6. Исправление выявленных дефектов

При проверке приложения были выявлены и устранены следующие дефекты.

Проблема с разделителем десятичных дробей. При вводе числа с запятой выражение не вычислялось, так как разборщик ожидал точку. Решение: в токенизаторе запятая заменяется на точку перед вызовом decimal.TryParse, а разбор выполняется в инвариантной культуре.

```
var numberText = expression.Substring(start, i - start).Replace(',', '.');
```

Невидимые кнопки панели инструментов. Кнопки «RU/EN» и «Тема» имели прозрачный фон и нулевую рамку, поэтому на тёмном фоне не отображались. Решение: заданы фон PanelBackground, рамка KeyBorderBrush и внутренние отступы.

Отсутствующий глиф клавиши удаления. Символ клавиши backspace не отображался в выбранном шрифте. Решение: клавиша заменена на символ ←.

Светлая системная рамка окна в тёмной теме. Решение: при загрузке окна и при каждой смене темы вызывается функция DwmSetWindowAttribute с атрибутом DWMWA_USE_IMMERSIVE_DARK_MODE, а признак тёмной темы определяется по яркости фона.

```
private void ApplyTitleBarTheme()
{
    var handle = new WindowInteropHelper(this).Handle;
    if (handle == IntPtr.Zero || DataContext is not MainViewModel vm)
        return;

    int useDarkMode = IsDarkColor(vm.PageBackground) ? 1 : 0;
    if (DwmSetWindowAttribute(handle, 20, ref useDarkMode, sizeof(int)) != 0)
        DwmSetWindowAttribute(handle, 19, ref useDarkMode, sizeof(int));
}
```

Тест, проверявший акцентный цвет по умолчанию. После смены палитры ожидаемое значение в тесте было обновлено с #2A8C6A на #5B7CFA.

5. Результаты работы

По итогам работы создано полнофункциональное WPF-приложение «Калькулятор Зеленина» с модульной архитектурой и покрытием модульными тестами.

Реализованный функционал:

- вычисление выражений с операторами +, −, ×, ÷, ^ и скобками;
- поддержка унарного минуса и десятичных дробей с точкой и запятой;
- предпросмотр результата без нажатия клавиши «=»;
- история вычислений с восстановлением выражения по щелчку и ограничением в 200 записей;
- светлая и тёмная темы с акцентным цветом #5B7CFA;
- переключение интерфейса RU/EN;
- ввод с клавиатуры: цифры, операторы, Enter, Backspace, Escape;
- сохранение истории и настроек между запусками.

Разработанные модульные тесты (42):

- базовые арифметические операции — 5;
- десятичные числа и разделители — 5;
- приоритет и ассоциативность — 5;

- унарный минус — 4;
- возведение в степень и граничные случаи — 4;
- обработка ошибок (деление на ноль, переполнение, некорректный ввод) — 9;
- история вычислений — 6;
- настройки темы — 4.

Внешний вид приложения в тёмной и светлой темах показан на рисунках 1 и 2.

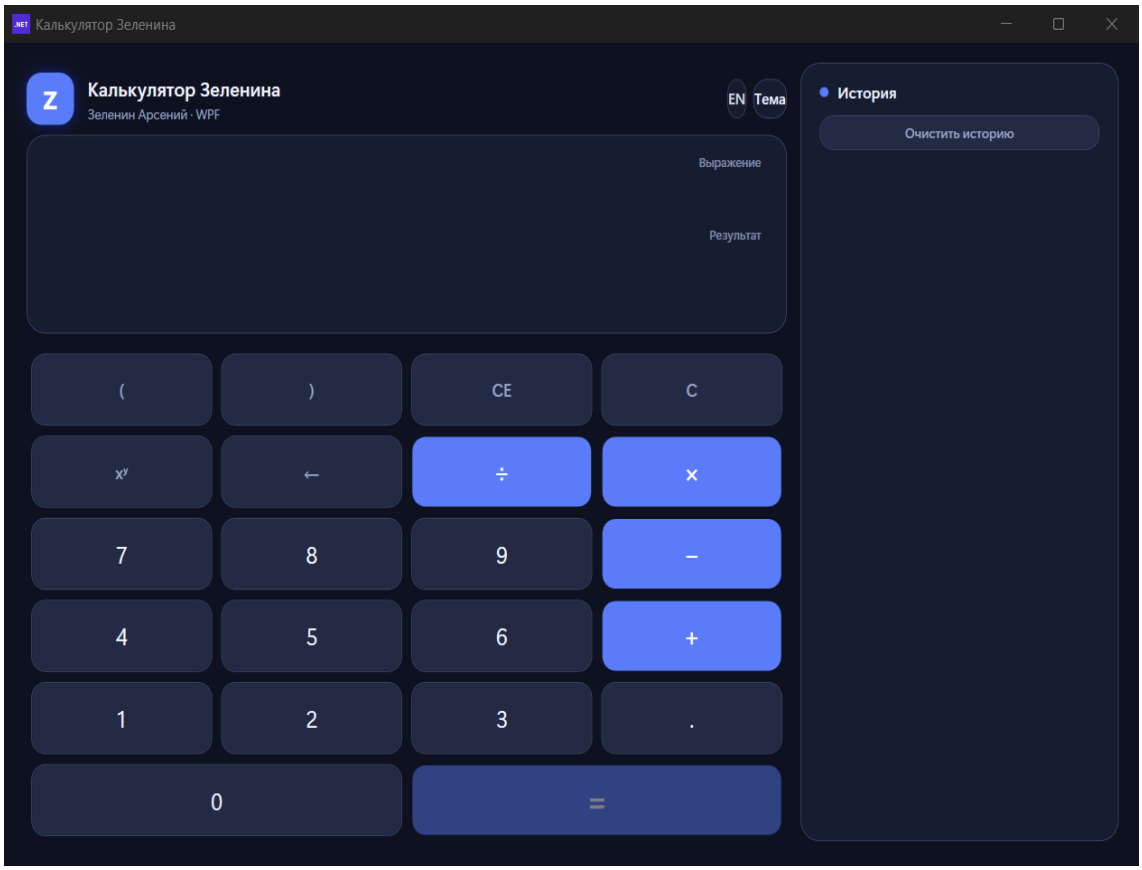


Рис. 1. Главное окно приложения в тёмной теме

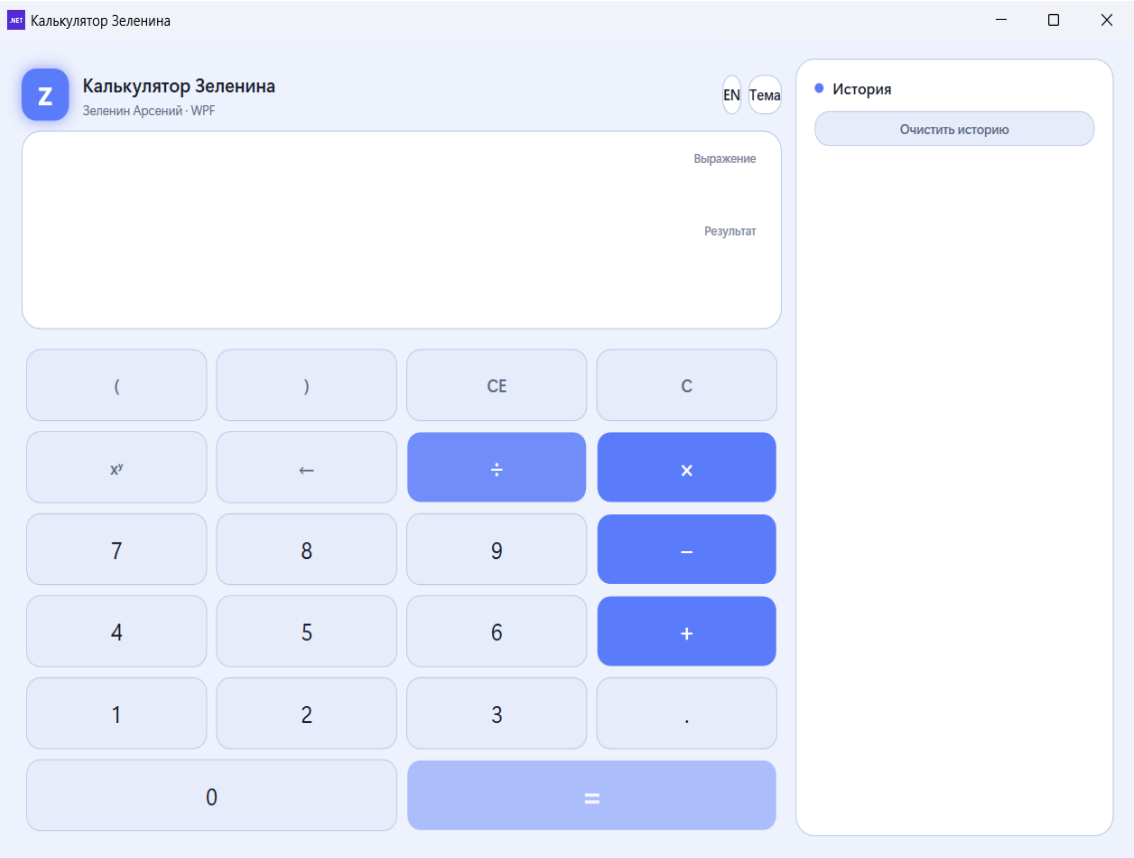


Рис. 2. Главное окно приложения в светлой теме

6. Заключение

Выполнение практической работы позволило закрепить следующие компетенции.

Технические навыки:

- Visual Studio: создание решения из нескольких проектов, настройка зависимостей, навигация по коду;
- C# и .NET: библиотеки классов, записи и перечисления, обработка исключений, работа с JSON;
- WPF: XAML, стили и шаблоны элементов управления, триггеры, привязка данных, команды, MVVM;
- модульное тестирование: xUnit, атрибуты [Fact], [Theory] и [InlineData], утверждения Assert, проверка исключений.

Методологические аспекты:

- разделение бизнес-логики и слоя представления;
- тестирование логики без запуска интерфейса;

- рефакторинг по результатам тестов и защита от регрессий.

Практическая значимость: модульное тестирование позволило проверить приоритет операций, обработку ошибок и сохранение данных, а также упростило изменение оформления приложения — правки дизайна не затрагивают логику и подтверждаются тестами.

Возможности расширения проекта:

- научный режим (тригонометрические функции, логарифмы);
- память калькулятора (M+, M−, MR, MC);
- экспорт истории в файл;
- поддержка комплексных чисел;
- настройка акцентного цвета в интерфейсе.

Вывод: проект продемонстрировал эффективность модульного подхода к разработке — изоляция бизнес-логики, покрытие тестами и чёткая архитектура позволяют создавать надёжные и поддерживаемые приложения на платформе .NET.